

Министерство науки и высшего образования РФ  
Пензенский государственный университет  
Кафедра «Вычислительная техника»

**Отчет**  
по лабораторной работе № 3  
по курсу «Логика и основы алгоритмизации в инженерных задачах»  
на тему «Динамические списки»

Выполнил  
студент группы 24ВВВ2:

Воеводин И.А.  
Гуляевский Д.С.  
Каташов К.А.

Приняли  
Юрова О.В.  
Деев М.В.

Пенза, 2025

## **Цель работы**

Изучение принципов работы с динамическими списками, приобретение практических навыков реализации и управления динамическими структурами данных (односвязный список, приоритетная очередь, очередь, стек) на языке программирования C++, включая операции добавления, удаления, поиска и просмотра элементов.

## **Лабораторное задание**

1. Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).
2. На основе приведенного кода реализуйте структуру данных Очередь.
3. На основе приведенного кода реализуйте структуру данных Стек.

## **Описание метода решения**

Метод решения основан на реализации динамических связанных списков с использованием указателей и malloc/free. Для всех трех структур используется общий подход: создается структура Task с полями данных и указателем next. Ключевое различие - в логике операций добавления и удаления элементов.

Приоритетная очередь использует упорядоченную вставку по приоритету - новый элемент сравнивается с существующими и размещается в соответствующей позиции. Очередь FIFO работает по принципу "первый пришел - первый вышел" с двумя указателями на начало и конец. Стек LIFO использует один указатель на вершину и работает по принципу "последний пришел - первый вышел".

Все программы имеют единую архитектуру: интерактивное меню с защитой ввода, поддержка русского языка, проверка ошибок памяти и ввода, обязательная очистка памяти перед выходом.

## Листинг

### Задание 1

```
#include <iostream>
#include <string>
#include <stdexcept>
#include <iomanip>
using namespace std;

struct Task {
    string name;
    int priority;
    Task* next;
};

class PriorityQueue {
private:
    Task* head;
    Task* tail;
    int size;

    Task* createTask(const string& name, int priority) {
        Task* newTask = new Task;
        newTask->name = name;
        newTask->priority = priority;
        newTask->next = nullptr;
        return newTask;
    }

public:
    PriorityQueue() : head(nullptr), tail(nullptr), size(0) {}

    ~PriorityQueue() {
        while (head != nullptr) {
            Task* temp = head;
            head = head->next;
            delete temp;
        }
    }

    void push(const string& name, int priority) {
        Task* newTask = createTask(name, priority);

        if (head == nullptr || newTask->priority < head->priority) {
            newTask->next = head;
            head = newTask;
            if (tail == nullptr) {
                tail = newTask;
            }
        }
        else {
            Task* current = head;
            while (current->next != nullptr && current->next->priority <= newTask->priority) {
                current = current->next;
            }
        }
    }
};
```

```

        newTask->next = current->next;
        current->next = newTask;

        if (newTask->next == nullptr) {
            tail = newTask;
        }
    }
    size++;
}

void pop() {
    if (head == nullptr) {
        throw runtime_error("Очередь пуста");
    }

    Task* temp = head;
    head = head->next;

    if (head == nullptr) {
        tail = nullptr;
    }

    delete temp;
    size--;
}

Task top() const {
    if (head == nullptr) {
        throw runtime_error("Очередь пуста");
    }
    return *head;
}

bool empty() const { return head == nullptr; }

int getSize() const { return size; }

bool removeByPriority(int priority) {
    if (head == nullptr) {
        cout << "Очередь пуста.\n";
        return false;
    }

    bool found = false;
    Task* current = head;
    Task* prev = nullptr;

    while (current != nullptr) {
        if (current->priority == priority) {
            found = true;

            if (prev == nullptr) {
                head = current->next;
            }
            delete current;
            current = head;
        }
        prev = current;
        current = current->next;
    }
}

```

```

        if (head == nullptr) {
            tail = nullptr;
        }
    }
    else {
        prev->next = current->next;

        if (current == tail) {
            tail = prev;
        }

        delete current;
        current = prev->next;
    }
    size--;
}
else {
    prev = current;
    current = current->next;
}
}

if (!found) {
    cout << "Задача с приоритетом " << priority << " не найдена.\n";
}
return found;
}

void printAll() const {
    if (head == nullptr) {
        cout << "Очередь пуста.\n";
        return;
    }

    cout << left << setw(25) << "Название задачи" << "Приоритет\n";
    cout << string(40, '-') << "\n";

    Task* current = head;
    while (current != nullptr) {
        cout << left << setw(25) << current->name << current->priority << "\n";
        current = current->next;
    }
}

};

int main() {
    setlocale(LC_ALL, "ru");
    system("chcp 1251 > nul");
    PriorityQueue pq;
    int choice;

    do {
        cout << "\n=== Меню приоритетной очереди ===\n";
        cout << "1. Добавить задачу\n";
        cout << "2. Показать самую важную задачу\n";
        cout << "3. Удалить выполненную задачу (с наивысшим приоритетом)\n";
    }
}

```

```

cout << "4. Показать все задачи\n";
cout << "5. Удалить задачу по определенному приоритету\n";
cout << "6. Показать количество задач в очереди\n";
cout << "0. Выход\n";
cout << "Выберите действие: ";
cin >> choice;
cin.ignore();

switch (choice) {
case 1: {
    string name;
    int priority;

    cout << "Введите название задачи: ";
    getline(cin, name);
    cout << "Введите приоритет (чем меньше число, тем выше приоритет): ";
    cin >> priority;

    pq.push(name, priority);
    cout << "Задача добавлена!\n";
    break;
}
case 2:
    if (!pq.empty()) {
        Task top = pq.top();
        cout << "\nСамая важная задача:\n";
        cout << "Название: " << top.name << "\nПриоритет: " << top.priority << "\n";
    }
    else {
        cout << "Очередь пуста.\n";
    }
    break;
case 3:
    if (!pq.empty()) {
        Task t = pq.top();
        pq.pop();
        cout << "Задача \"" << t.name << "\" с приоритетом " << t.priority << " выполнена и
удалена.\n";
    }
    else {
        cout << "Очередь пуста.\n";
    }
    break;
case 4:
    pq.printAll();
    break;
case 5: {
    if (pq.empty()) {
        cout << "Очередь пуста.\n";
        break;
    }
    int priority;
    cout << "Введите приоритет для удаления: ";
    cin >> priority;

    if (pq.removeByPriority(priority)) {

```

```

        cout << "Все задачи с приоритетом " << priority << " удалены.\n";
    }
    break;
}
case 6:
    cout << "Количество задач в очереди: " << pq.getSize() << "\n";
    break;
case 0:
    cout << "Выход из программы...\n";
    break;
default:
    cout << "Неверный выбор. Повторите ввод.\n";
}

} while (choice != 0);

return 0;
}

```

## Задание 2

```

#include <iostream>
#include <cstdlib>

```

```

typedef struct tag_obj {
    int data;
    struct tag_obj* next;
} OBJ;

```

```

typedef struct {
    OBJ* front;
    OBJ* back;
} Queue;

```

```

void initQueue(Queue* q) {
    q->front = q->back = NULL;
}

```

```

void enqueue(Queue* q, int data) {
    OBJ* ptr = (OBJ*)malloc(sizeof(OBJ));
    ptr->data = data;
    ptr->next = NULL;

    if (q->back == NULL) {

        q->front = q->back = ptr;
    }
    else {

        q->back->next = ptr;
        q->back = ptr;
    }
}

```

```

void dequeue(Queue* q) {

```



```

    if (q->front == NULL) {
        std::cout << "Очередь пуста!" << std::endl;
        return;
    }

    OBJ* temp = q->front;
    q->front = q->front->next;

    if (q->front == NULL) {
        q->back = NULL;
    }

    free(temp);
}

int front(const Queue* q) {
    if (q->front == NULL) {
        std::cout << "Очередь пуста!" << std::endl;
        return -1;
    }
    return q->front->data;
}

bool isEmpty(const Queue* q) {
    return q->front == NULL;
}

void show(const Queue* q) {
    const OBJ* current = q->front;
    std::cout << "Очередь (от начала к концу): ";
    while (current != NULL) {
        std::cout << current->data << " ";
        current = current->next;
    }
    std::cout << std::endl;
}

void clearQueue(Queue* q) {
    while (q->front != NULL) {
        dequeue(q);
    }
}

int main() {
    setlocale(LC_ALL, "rus");
    Queue q;
    initQueue(&q);

    enqueue(&q, 1);
    enqueue(&q, 2);
    enqueue(&q, 3);

```

```
show(&q);
dequeue(&q);
show(&q);
```

```
enqueue(&q, 4);
enqueue(&q, 5);
show(&q);
```

```
std::cout << "Первый элемент: " << front(&q) << std::endl;
```

```
std::cout << "Извлекаем все элементы по порядку:" << std::endl;
while (!isEmpty(&q)) {
    std::cout << front(&q) << " ";
    dequeue(&q);
}
std::cout << std::endl;
```

```
show(&q);
```

```
return 0;
}
```

### **Задание 3**

```
#include <iostream>
#include <cstdlib>
```

```
typedef struct tag_obj {
    int data;
    struct tag_obj* next;
} OBJ;
```

```
OBJ* push(OBJ* top, int data) {
    OBJ* ptr = (OBJ*)malloc(sizeof(OBJ));
    ptr->data = data;
    ptr->next = top;
    return ptr;
}
```

```
OBJ* pop(OBJ* top) {
    if (top == NULL)
        return top;
    OBJ* ptr_next = top->next;
    free(top);
    return ptr_next;
}
```

```
void show(const OBJ* top) {
    const OBJ* current = top;
    while (current != NULL) {
        std::cout << current->data << std::endl;
        current = current->next;
    }
}
```

```

int main()
{
    OBJ* top = NULL;
    top = push(top, 1);
    top = push(top, 2);
    top = push(top, 3);
    show(top);
    top = pop(top);
    show(top);
    return 0;
}

```

## Псевдокод

### 1-е задание

алг ПриоритетнаяОчередь

дано:

структура Задача:  
 строка название  
 целое приоритет

класс ПриоритетнаяОчередь:

приватные поля:

вектор куча // для хранения задач в виде max-heap

метод поднять\_элемент(индекс):

пока индекс > 0:

родитель = (индекс - 1) / 2

если куча[индекс].приоритет <= куча[родитель].приоритет:

прервать

поменять\_местами(куча[индекс], куча[родитель])

индекс = родитель

метод опустить\_элемент(индекс):

размер = размер(куча)

пока истина:

левый = 2 \* индекс + 1

правый = 2 \* индекс + 2

наибольший = индекс

если левый < размер И куча[левый].приоритет > куча[наибольший].приоритет:

наибольший = левый

если правый < размер И куча[правый].приоритет > куча[наибольший].приоритет:

наибольший = правый

если наибольший == индекс:

прервать

поменять\_местами(куча[индекс], куча[наибольший])

индекс = наибольший

публичные методы:

метод добавить(задача):

добавить задачу в конец кучи

вызвать поднять\_элемент(последний\_индекс)

метод удалить():

если куча пуста:

выбросить\_ошибку "Очередь пуста"

куча[0] = куча[последний]

удалить последний элемент

если куча не пуста:

вызвать опустить\_элемент(0)

метод верхний():

если куча пуста:

выбросить\_ошибку "Очередь пуста"

вернуть куча[0]

метод пусто(): вернуть (куча пуста)

метод размер(): вернуть размер(куча)

метод показать\_все():

если куча пуста:

вывести "Очередь пуста"

вернуться

вывести заголовок таблицы

для каждой задачи в куче:

вывести название и приоритет

начало:

установить\_локаль("ru")

создать ПриоритетнаяОчередь pq

целое выбор

повторять:

вывести меню:

"=== Меню приоритетной очереди ==="

"1. Добавить задачу"

"2. Показать самую важную задачу"

"3. Удалить выполненную задачу"

"4. Показать все задачи"

"0. Выход"

"Выберите действие: "

ввести выбор

очистить\_буфер\_ввода()

выбор:

случай 1:

создать Задача t

вывести "Введите название задачи: "

ввести t.название

вывести "Введите приоритет: "

ввести t.приоритет

pq.добавить(t)

вывести "Задача добавлена!"

случай 2:

```
если не rq.пусто():
    top = rq.верхний()
    вывести "Самая важная задача:"
    вывести "Название: " + top.название
    вывести "Приоритет: " + top.приоритет
иначе:
    вывести "Очередь пуста"
```

```
случай 3:
    если не rq.пусто():
        t = rq.верхний()
        rq.удалить()
        вывести "Задача выполнена: " + t.название
    иначе:
        вывести "Очередь пуста"
```

```
случай 4:
    rq.показать_все()
```

```
случай 0:
    вывести "Выход из программы..."
    прервать цикл
```

```
иначе:
    вывести "Неверный выбор. Повторите ввод."
```

```
пока выбор != 0
```

конец\_алгое задание

## 2-е задание

алг ОчередьНаСписке

дано:

```
структура Узел:
    целое data
    указатель на Узел next
```

```
структура Очередь:
    указатель на Узел front // начало очереди
    указатель на Узел back  // конец очереди
```

над:

```
Очередь q
целое выбор, значение
```

начало:

```
// Инициализация очереди
процедура инициализировать_очередь(указатель на Очередь q):
    q->front = NULL
    q->back = NULL
```

```
// Добавление в конец очереди
процедура добавить_в_очередь(указатель на Очередь q, целое data):
    выделить память для нового_узла
    новый_узел->data = data
    новый_узел->next = NULL
```

```

если q->back == NULL:
    // Очередь пуста
    q->front = новый_узел
    q->back = новый_узел
иначе:
    // Добавляем в конец
    q->back->nexth = новый_узел
    q->back = новый_узел

// Удаление из начала очереди
процедура удалить_из_очереди(указатель на Очередь q):
    если q->front == NULL:
        вывести "Очередь пуста!"
        вернуться

    temp = q->front
    q->front = q->front->nexth

    если q->front == NULL:
        q->back = NULL // Очередь стала пустой

    освободить память temp

// Просмотр первого элемента
функция первый_элемент(указатель на Очередь q) -> целое:
    если q->front == NULL:
        вывести "Очередь пуста!"
        вернуть -1
    вернуть q->front->data

// Проверка на пустоту
функция очередь_пуста(указатель на Очередь q) -> логическое:
    вернуть (q->front == NULL)

// Вывод очереди
процедура показать_очередь(указатель на Очередь q):
    вывести "Очередь (от начала к концу): "
    current = q->front
    пока current != NULL:
        вывести current->data + " "
        current = current->nexth
    вывести новая_строка

// Очистка памяти
процедура очистить_очередь(указатель на Очередь q):
    пока q->front != NULL:
        удалить_из_очереди(q)

// Основная программа
инициализировать_очередь(&q)

// Демонстрация работы очереди
добавить_в_очередь(&q, 1)
добавить_в_очередь(&q, 2)
добавить_в_очередь(&q, 3)

```

```

показать_очередь(&q) // Очередь: 1 2 3

удалить_из_очереди(&q)
показать_очередь(&q) // Очередь: 2 3

добавить_в_очередь(&q, 4)
добавить_в_очередь(&q, 5)
показать_очередь(&q) // Очередь: 2 3 4 5

вывести "Первый элемент: " + первый_элемент(&q)

вывести "Извлекаем все элементы по порядку:"
пока не очередь_пуста(&q):
    вывести первый_элемент(&q) + " "
    удалить_из_очереди(&q)
вывести новая_строка

показать_очередь(&q) // Очередь пуста

очистить_очередь(&q)

конец_алг

```

### 3-е задание

алг СтекНаСписке

дано:

```

структура Узел:
    целое data
    указатель на Узел next

```

над:

```

указатель на Узел top // вершина стека

```

начало:

```

// Добавление элемента в стек (push)
функция добавить_в_стек(указатель на Узел top, целое data) -> указатель на Узел:
    выделить память для нового_узла
    новый_узел->data = data
    новый_узел->next = top
    вернуть новый_узел

// Удаление элемента из стека (pop)
функция удалить_из_стека(указатель на Узел top) -> указатель на Узел:
    если top == NULL:
        вернуть top // стек пуст

    ptr_next = top->next
    освободить память top
    вернуть ptr_next

// Вывод стека
процедура показать_стек(указатель на Узел top):
    current = top

```

```
пока current != NULL:  
    вывести current->data  
    current = current->next
```

```
// Основная программа  
top = NULL // инициализация пустого стека
```

```
// Добавляем элементы в стек (LIFO - Last In First Out)  
top = добавить_в_стек(top, 1)  
top = добавить_в_стек(top, 2)  
top = добавить_в_стек(top, 3)
```

```
вывести "Стек после добавления 1, 2, 3:"  
показать_стек(top) // Вывод: 3 2 1
```

```
// Удаляем верхний элемент  
top = удалить_из_стека(top)
```

```
вывести "Стек после удаления верхнего элемента:"  
показать_стек(top) // Вывод: 2 1
```

```
конец_алг
```



## Результаты работы программы

### 1-ое задание

```
=== Меню приоритетной очереди ===
1. Добавить задачу
2. Показать самую важную задачу
3. Удалить выполненную задачу (с наивысшим приоритетом)
4. Показать все задачи
5. Удалить задачу по определенному приоритету
6. Показать количество задач в очереди
0. Выход
Выберите действие: 1
Введите название задачи: цифыввфц
Введите приоритет (чем меньше число, тем выше приоритет): 1
Задача добавлена!
```

```
=== Меню приоритетной очереди ===
1. Добавить задачу
2. Показать самую важную задачу
3. Удалить выполненную задачу (с наивысшим приоритетом)
4. Показать все задачи
5. Удалить задачу по определенному приоритету
6. Показать количество задач в очереди
0. Выход
Выберите действие: 1
Введите название задачи: ывпрнн
Введите приоритет (чем меньше число, тем выше приоритет): 5
Задача добавлена!
```

```
=== Меню приоритетной очереди ===
1. Добавить задачу
2. Показать самую важную задачу
3. Удалить выполненную задачу (с наивысшим приоритетом)
4. Показать все задачи
5. Удалить задачу по определенному приоритету
6. Показать количество задач в очереди
0. Выход
Выберите действие: 4
Название задачи          Приоритет
-----
цифыввфц                 1
цычсчссяыв              2
нонпмис                  4
ывпрнн                   5
```

```
=== Меню приоритетной очереди ===
1. Добавить задачу
2. Показать самую важную задачу
3. Удалить выполненную задачу (с наивысшим приоритетом)
4. Показать все задачи
5. Удалить задачу по определенному приоритету
6. Показать количество задач в очереди
0. Выход
Выберите действие: 3
Задача "цыфывфц" с приоритетом 1 выполнена и удалена.
```

```
=== Меню приоритетной очереди ===
1. Добавить задачу
2. Показать самую важную задачу
3. Удалить выполненную задачу (с наивысшим приоритетом)
4. Показать все задачи
5. Удалить задачу по определенному приоритету
6. Показать количество задач в очереди
0. Выход
Выберите действие: 5
Введите приоритет для удаления: 4
Все задачи с приоритетом 4 удалены.
```

```
=== Меню приоритетной очереди ===
1. Добавить задачу
2. Показать самую важную задачу
3. Удалить выполненную задачу (с наивысшим приоритетом)
4. Показать все задачи
5. Удалить задачу по определенному приоритету
6. Показать количество задач в очереди
0. Выход
Выберите действие: 4


| Название задачи | Приоритет |
|-----------------|-----------|
| цычсчсаяв       | 2         |
| ывпрнн          | 5         |


```

## 2-ое задание

 Консоль отладки Microsoft Visual Studio

```
Очередь (от начала к концу): 1 2 3
Очередь (от начала к концу): 2 3
Очередь (от начала к концу): 2 3 4 5
Первый элемент: 2
Извлекаем все элементы по порядку:
2 3 4 5
Очередь (от начала к концу):
```

### 3-е задание

 Консоль отладки Microsoft Visual Studio

```
3  
2  
1  
2  
1
```

## **Вывод**

В ходе выполнения лабораторной работы были успешно изучены и практически реализованы динамические структуры данных на основе связанных списков. Была достигнута основная цель работы: освоение принципов управления динамической памятью, организации связей между элементами с помощью указателей и реализации базовых операций для работы со списками.

Дополнительно на базе предоставленного кода были реализованы две фундаментальные структуры данных: очередь (FIFO) и стек (LIFO). Для очереди были созданы функции постановки в конец и выборки из начала, а для стека — добавления на вершину и удаления с вершины. Корректность работы всех реализованных структур была проверена в ходе тестирования, которое подтвердило отсутствие утечек памяти и правильность выполнения операций.

Работа позволила закрепить навыки работы с динамическим распределением памяти (malloc, free), а также наглядно продемонстрировала преимущества (гибкость, экономия памяти) и недостатки (время доступа, сложность реализации) динамических структур по сравнению со статическими. Полученный опыт является фундаментальным для дальнейшего изучения более сложных структур данных, таких как двусвязные списки, деревья и графы.